

ft_ascii_caster

Le Raycasting à l'ancienne : Wolfenstein 3D dans votre terminal C

Résumé : Ce projet consiste à réaliser un moteur graphique 3D en mode texte (ASCII) fonctionnant directement dans le terminal UNIX. En utilisant la technique du Raycasting, votre programme devra charger une carte, gérer les déplacements d'un joueur en temps réel et afficher une perspective tridimensionnelle sans aucune bibliothèque graphique externe.

1. Objectifs de l'exercice

Ce projet s'inspire librement des célèbres projets `cub3d` et `minishell`. L'objectif caché derrière ce sujet est de tester la capacité à concevoir et structurer un projet complexe en mode **vibecoding** (génération et pilotage par IA). Les moteurs graphiques et la gestion brute du terminal forcent l'utilisation de notions mathématiques couplées à des configurations système pointues, idéal pour évaluer l'interaction homme-IA sur une architecture complète.

2. Consignes Générales & Contraintes

- Le projet doit être entièrement codé en **Langage C**.
- Votre code doit compiler avec les flags réglementaires : `-Wall -Wextra -Werror`.
- Zéro fuite de mémoire :** Tout octet alloué dynamiquement (`malloc`) doit être libéré proprement en cas de succès, mais surtout en cas d'erreur ou d'interruption.
- Fonctions autorisées :** `open`, `close`, `read`, `write`, `malloc`, `free`, `perror`, `strerror`, `exit`, `usleep`, ainsi que les fonctions de `<math.h>` (`sin`, `cos`, `tan`, `sqrt`) et de `<termios.h>`.
- Toute bibliothèque graphique (MiniLibX, OpenGL, ncurses) est **strictement interdite**. Le rendu doit se faire via des écritures standard (`write` ou `printf`) en utilisant des codes d'échappement ANSI pour rafraîchir l'écran.

3. Partie Mandatoire

Votre programme devra se nommer `ft_ascii_caster` et accepter un unique argument : le chemin vers un fichier de carte se terminant par l'extension `.map`.

```
./ft_ascii_caster maps/classic.map
```

3.1. Parsing de la Carte (.map)

Le fichier de configuration de la carte est un fichier texte brut représentant la grille vue du dessus. Les caractères autorisés sont :

- `1` : Un mur.

- **0** : Un espace vide sur lequel le joueur peut marcher.
- **N**, **S**, **E**, ou **W** : La position initiale du joueur et sa direction de regard (Nord, Sud, Est, Ouest).

Exemple de carte valide :

```
1111111111
1000000001
1011001101
10000N0001
1111111111
```

Contrainte de Parsing :

Le parser doit être intraitable. Si la carte n'est pas fermée par des murs (les bords contiennent un **0**), s'il y a plusieurs joueurs, si un caractère est invalide ou s'il y a des lignes vides au milieu, le programme doit afficher **Error** suivi d'un message explicite, tout libérer, et quitter proprement.

3.2. Mode Non-Bloquant et Configuration du Terminal

Pour jouer en temps réel, vous devez capturer les touches du clavier (**W**, **A**, **S**, **D** pour se déplacer, et les flèches ou touches de rotation) **immédiatement**, sans que l'utilisateur n'ait besoin d'appuyer sur la touche *Entrée*. Vous devez utiliser la structure `struct termios` pour passer le terminal en mode raw (non canonique) et configurer une lecture non-bloquante.

3.3. Le Moteur de Raycasting & Affichage ASCII

À chaque itération de votre boucle principale :

1. Vous devez récupérer la taille courante du terminal ou fixer une résolution par défaut (ex: 80 colonnes par 40 lignes).
2. Pour chaque colonne de l'écran, vous lancez un rayon optique virtuel depuis la position du joueur avec un angle dépendant de son champ de vision (FOV de ~60°).
3. Vous utilisez l'algorithme **DDA (Digital Differential Analysis)** pour calculer l'intersection exacte du rayon avec le premier mur **1** de la grille.
4. En fonction de la distance calculée, vous déterminez la hauteur apparente du mur à afficher sur cette colonne pour créer l'effet de perspective 3D.
5. **Ombrage ASCII** : Pour accentuer l'effet 3D, le caractère utilisé pour dessiner le mur doit dépendre de la distance. Plus le mur est proche, plus le caractère doit être "dense" (ex: **@** pour très proche, **#**, **0**, **x**, **.** pour le très lointain). Les espaces vides au-dessus et en-dessous représenteront le plafond et le sol.

4. Partie Bonus

Les bonus ne seront analysés que si la partie mandataire fonctionne de manière irréprochable.

- **Gestion des Collisions** : Empêcher le joueur de traverser les murs lorsqu'il se déplace. Il doit glisser contre le mur ou s'arrêter net.
- **Orientation des Faces (Textures ASCII)** : Afficher des caractères différents selon la face du mur touchée par le rayon (ex: **N** pour un mur Nord, **S** pour un mur Sud, etc.).

- **Mini-carte intégrée** : Afficher une vue 2D simplifiée de la carte dans un coin supérieur de l'écran avec un curseur indiquant la position et l'orientation du joueur en temps réel.